

**/\* This note has been compiled from various sources and can be taken as reference only \*/**

## **Chapter 7: Hypervisor and Virtual Systems [4 Hrs]**

### **#1. Virtualization:**

**Virtualization** is the process of creating a **virtual version** of computing resources such as hardware platforms, storage devices, operating systems, or networks.

It allows multiple **virtual machines (VMs)** to run on a single **physical machine** by abstracting and dividing the system's physical resources.

Virtualization enables one physical machine to behave like **many independent computers**, each with its own OS and applications.

### **# Key Components of Virtualization**

| <b>Component</b>          | <b>Description</b>                               |
|---------------------------|--------------------------------------------------|
| <b>Host Machine</b>       | The physical machine where virtualization runs   |
| <b>Guest Machine (VM)</b> | The virtual system running inside the host       |
| <b>Hypervisor (VMM)</b>   | Software that manages and runs virtual machines  |
| <b>Virtual Resources</b>  | Simulated CPU, memory, disk, network for each VM |

### **# Types of Virtualization**

#### **a. Hardware Virtualization**

- Simulates physical hardware to run multiple operating systems.
- Example: Running Windows and Linux on the same machine.

#### **b. Operating System Virtualization**

- Allows multiple isolated user-space instances (containers) on a single OS kernel.
- Example: Docker containers on Linux

#### **c.Storage Virtualization**

- Combines physical storage from multiple devices into one logical unit.

#### **d. Network Virtualization**

- Abstracts physical networking devices to create virtual networks (e.g., VLANs, SDN(Software Defined Network)).

## # Real-world Example of Virtualization- Imp. To understand

### Scenario:

A software development company has one powerful server. They want to test their app on both **Windows** and **Linux** environments.

### Without Virtualization:

They need two separate computers for Windows and Linux → **Costly** and **space-consuming**.

### With Virtualization:

They install a **hypervisor** like **VMware** or **VirtualBox** on the server, and then run:

- One **Windows VM**
  - One **Linux VM**
- Both use shared CPU, RAM, and disk resources.

This allows them to:

- Run multiple OSs simultaneously
- Save costs
- Easily take snapshots or backups

## # Common Virtualization Software

| Software              | Type                 | Platform (runs in) |
|-----------------------|----------------------|--------------------|
| VirtualBox            | Hosted Hypervisor    | Cross-platform     |
| VMware Workstation    | Hosted Hypervisor    | Windows/Linux      |
| KVM (Kernel-based VM) | Bare-metal           | Linux              |
| Microsoft Hyper-V     | Bare-metal           | Windows            |
| Docker                | OS-level (container) | Linux/Windows      |

## # Why Use Virtualization?-Imp

- **Efficient hardware utilization**
- **Cost reduction** (fewer physical machines)
- **Isolation** of applications or services
- **Testing and development** environments
- **Scalability and flexibility** in enterprise and cloud systems

## # Possible Questions

1. What is virtualization and how does it benefit system administration?
  2. Differentiate between host and guest in virtualization.
  3. Give a real-life example of where virtualization can be useful.
  4. What is the role of a hypervisor?
- 

## #2. Hypervisor

A **Hypervisor** (also called a **Virtual Machine Monitor – VMM**) is system software that creates and manages **virtual machines (VMs)** by abstracting hardware resources and distributing them to multiple guest operating systems.

Hypervisors allow multiple OSs to run on a single physical machine independently.

## #Types of Hypervisors

Hypervisors are categorized into **two main types** based on how they interact with the hardware:

### a. Type 1 Hypervisor (Bare-metal Hypervisor)

A **Type 1 hypervisor** runs **directly on the physical hardware** of the host system. It **does not require a host operating system**.

### Characteristics:

- High performance and efficiency
- Used in enterprise and cloud environments
- More secure and stable than Type 2
- Direct access to hardware

### Examples:

- **VMware ESXi**
- **Microsoft Hyper-V** (when installed without Windows Server GUI)
- **Xen**
- **KVM (Kernel-based Virtual Machine)**

### Real-life Use Case:- Need to Understand

In a **data center**, a server runs **VMware ESXi** (Type 1 hypervisor). Multiple VMs like Linux and Windows Server are deployed for clients and internal use.

### b. Type 2 Hypervisor (Hosted Hypervisor)

A **Type 2 hypervisor** runs **on top of a host operating system**, just like a normal software application.

#### Characteristics:

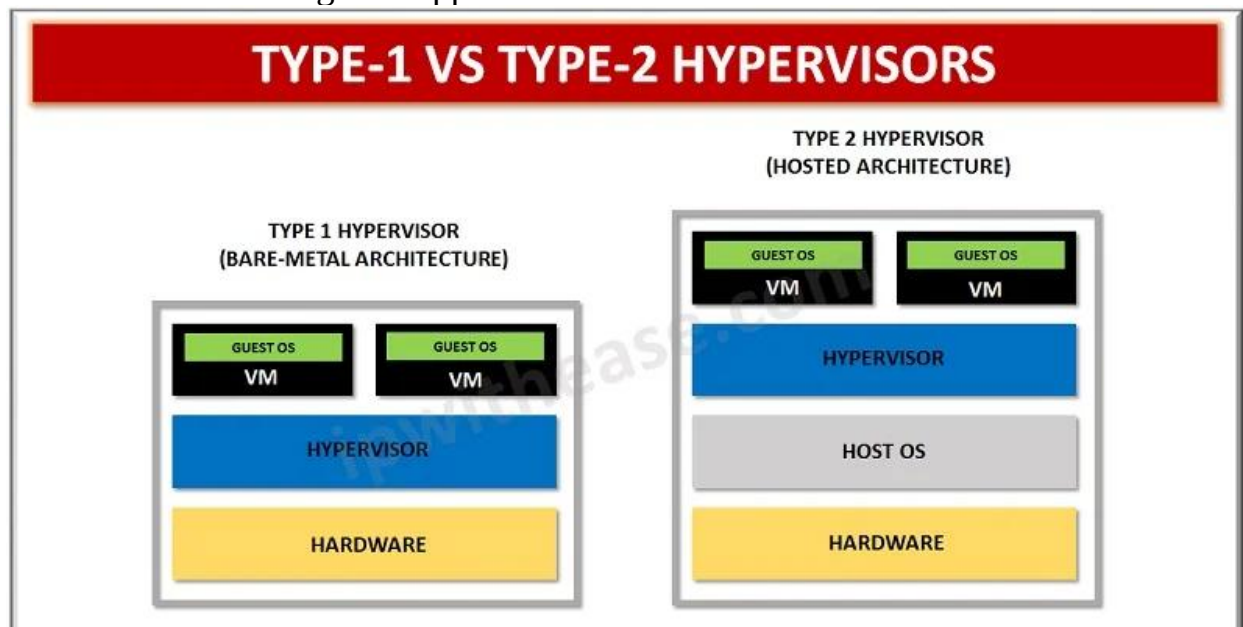
- Easier to install and use
- Suitable for personal computers and development
- Relies on host OS for hardware access → less efficient

#### Examples:

- **VirtualBox**
- **VMware Workstation**
- **Parallels Desktop (macOS)**
- **QEMU (when used in hosted mode)**

#### Real-life Use Case:

A developer using **Windows 11** installs **VirtualBox** (Type 2 hypervisor) to run a **Linux VM** for testing web applications.



#### Comparison Table

| Feature       | Type 1 Hypervisor         | Type 2 Hypervisor              |
|---------------|---------------------------|--------------------------------|
| Also known as | Bare-metal hypervisor     | Hosted hypervisor              |
| Runs on       | Directly on hardware      | On top of host OS              |
| Performance   | High                      | Moderate                       |
| Use case      | Servers, data centers     | Personal use, development      |
| Examples      | VMware ESXi, Hyper-V, KVM | VirtualBox, VMware Workstation |

---

### #3. Creating and Managing Virtual Machine with Qemu, Virtual Box and VMware – Covered in LAB

#### 4. Containerization

**Containerization** is **OS-based virtualization** that creates multiple virtual units in the **userspace**, known as **Containers**. **Containers** share the same host **kernel** but are isolated from each other through private **namespaces** and **resource control mechanisms** at the **OS level**.

**Container-based Virtualization** provides a different level of **abstraction** in terms of virtualization and isolation when compared with **hypervisors**. **Hypervisors** use a lot of **hardware** which results in **overhead** in terms of virtualizing **hardware** and **virtual device drivers**. A full **operating system** (e.g -**Linux**, **Windows**) runs on top of this virtualized hardware in each **virtual machine instance**.

But in contrast, **containers** implement **isolation of processes at the operating system level**, thus avoiding such overhead(use of lots of hardwares). These containers run on top of the same shared **operating system kernel** of the underlying host machine and one or more **processes** can be run within each **container**.

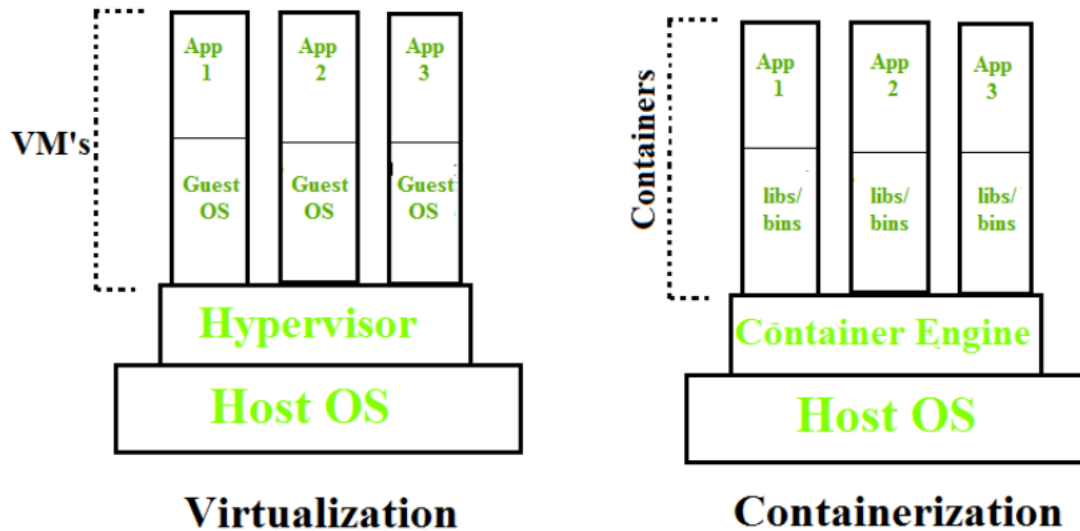
In **containers** you don't have to pre-allocate any **RAM**, it is allocated dynamically during the creation of **containers** while in **VMs** you need to first pre-allocate the **memory** and then create the **virtual machine**. **Containerization** has better **resource utilization** compared to **VMs** and a short **boot-up process**. **It is the next evolution in virtualization.**

**Containers** can run virtually anywhere, greatly easy **development** and **deployment**: on **Linux**, **Windows**, and **Mac operating systems**; on **virtual machines** or **bare metal**, on a developer's machine or in **data centers** on-premises; and of course, in the **public cloud**.

**Containers** virtualize **CPU**, **memory**, **storage**, and **network resources** at the **OS level**, providing developers with a **sandboxed view** of the **OS** logically isolated from other **applications**.

**Containerization** is a lightweight form of virtualization that allows applications to run in **isolated environments called containers**, sharing the host OS kernel but maintaining their own filesystem, libraries, and dependencies.

Unlike virtual machines, containers do **not require a full OS** inside them — they are faster, lighter, and more portable.



### 1. Docker- Imp



**Docker** is the **containerization platform** that is used to **package your application** and all its **dependencies** together in the form of **containers** to make sure that your application works seamlessly in any environment which can be developed or tested or in **production**. Docker is a tool designed to make it easier to **create, deploy, and run applications** by using containers. It was launched in **2013** by a company called **Dotcloud, Inc** which was later renamed **Docker, Inc**.

## 2. Kubernetes

**Kubernetes** (often called **K8s**) is an open-source **container orchestration platform** developed by Google. It manages the **deployment, scaling, and monitoring** of containerized applications across a cluster of machines.

Docker creates and runs containers; Kubernetes manages **many containers across many machines**.

Kubernetes is an container orchestration platform by which you can automate the deployment of the application , scaling of the application by depending on the traffic. Containers are light in weight which can be portable very easily from one server to another server very easily following makes ideal for running containerized applications in production.

1. Load Balancing
2. Service Discovery
3. Self- Management(healing)
4. Horizontal scaling

### ***#Comparison of Docker and Kubernetes***

| Feature     | Docker                           | Kubernetes                    |
|-------------|----------------------------------|-------------------------------|
| Purpose     | Run and manage single containers | Manage clusters of containers |
| Abstraction | Container                        | Pod (group of containers)     |
| Scale       | One machine (usually)            | Multiple machines (cluster)   |

### **Possible Questions**

1. What is a container and how is it different from a virtual machine?
2. What does Docker do in the context of containerization?
3. Explain the role of Kubernetes in managing containers.
4. Give one real-world example where containerization is useful.

## 5. Virtualization in Windows

**Virtualization in Windows** enables users to run multiple isolated operating systems or environments on a single physical computer. This improves **efficiency, testing capabilities, security, and resource management**.

Windows offers built-in virtualization features as well as compatibility with third-party tools, supporting various use cases — from professional development to enterprise-grade VM deployment.

### 1. Hyper-V: Built-in Hypervisor

- **Hyper-V** is a **Type 1 (bare-metal) hypervisor** developed by Microsoft.
- Available on **Windows 10/11 Pro, Enterprise, and Windows Server** editions.
- Enables users to run **multiple virtual machines (VMs)** on a single host.

#### Features:

- Full OS virtualization (Windows, Linux, etc.)
- Virtual networking (Virtual Switch Manager)
- Live migration (on supported systems)
- Snapshot/checkpoint support

#### Enabling Hyper-V (via PowerShell):

powershell

```
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V -All
```

#### Hyper-V Tools:

- **Hyper-V Manager:** GUI for managing VMs.
- **PowerShell Cmdlets:** e.g., New-VM, Start-VM, Get-VM.

#### Example:

powershell

```
New-VM -Name "UbuntuVM" -MemoryStartupBytes 2GB -Generation 2 -  
NewVHDPATH "C:\VMs\ubuntu.vhdx" -NewVHDSIZEBytes 40GB
```



## 2. PowerShell in Virtualization

**PowerShell** is a powerful automation and scripting tool built into Windows, widely used for:

- Automating Hyper-V VM creation and management
- Configuring virtual networks
- Managing containers and virtual environments

### Useful PowerShell Cmdlets for Hyper-V:- Just explain

| Cmdlet        | Description      |
|---------------|------------------|
| Get-VM        | Lists all VMs    |
| Start-VM      | Starts a VM      |
| Stop-VM       | Stops a VM       |
| Checkpoint-VM | Takes a snapshot |
| Remove-VM     | Deletes a VM     |

PowerShell makes **repeatable and automated** VM configuration possible — a key skill in DevOps and system administration.

## 3. WSL (Windows Subsystem for Linux)

**WSL** allows users to run a **Linux environment natively on Windows**, without needing a virtual machine or dual boot. It is a form of **lightweight OS-level virtualization**.

### Features:

- Access Linux command-line tools (bash, grep, apt, etc.)
- Share files and clipboard with Windows
- Run Linux scripts and applications
- Supports both **WSL 1 (translation layer)** and **WSL 2 (true Linux kernel)**

### Enabling WSL (PowerShell):

powershell

wsl --install

Or manually:

powershell

dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart

### Example:

Run Ubuntu on Windows:

bash

wsl --install -d Ubuntu

With WSL 2, Windows uses a lightweight VM to run the Linux kernel with improved performance and compatibility.

## 4. Windows Sandbox

- A **temporary, isolated environment** that lets users run untrusted apps or files safely.
- Each time it runs, it creates a **fresh Windows instance** — all changes are discarded after exit.
- **Lightweight** alternative to full VMs for quick testing.

### Enabling Sandbox (via PowerShell):

powershell

```
Enable-WindowsOptionalFeature -FeatureName "Containers-DisposableClientVM" -All -Online
```

## Benefits of Windows Virtualization

- Easy testing and development of multi-OS apps
- Secure and isolated environments
- Resource efficiency and hardware optimization
- DevOps-friendly (with PowerShell and WSL integration)

## End of Chapter 7

|                                       |                                          |
|---------------------------------------|------------------------------------------|
| Chapter 8 Overview of Contemporary OS | [ 3 Hrs]                                 |
| Presentation                          | Roll No (ends with)                      |
| 8.1 Windows and Linux based OS        | 1-41,2-42,3-43 (5 minutes for one group) |
| 8.2 Embedded and Mobile OS            | 4-44,5-45                                |
| 8.3 IOT and RT (Real Time) OS         | 6-46,7-47                                |
| 8.4 Robot and Smart Card OS           | 8-48,9-39,10-40                          |

